

# Feedback-Raster

## Selbstcheck und Dozenten-Rückmeldung zum Projekt

### Hinweis

Dieses Dokument hat zwei Zwecke. Sie können es zur **Selbstkontrolle** verwenden, etwa an Tag 5 oder Tag 6, um zu prüfen, wo Sie stehen. Und es ist die Grundlage für das **Feedback**, das Sie nach der Präsentation bekommen.

Es gibt keine Note. Die Prozentangaben benennen, welche Aspekte beim Feedback wie stark gewichtet werden.

### Pflichtumsetzung (40%)

**Frage:** Sind alle sechs Pflichtanforderungen sichtbar umgesetzt? Läuft die Anwendung aus dem ZIP heraus stabil?

| Kriterium                                                                                        | Erfüllt | Bemerkung |  |
|--------------------------------------------------------------------------------------------------|---------|-----------|--|
| Frontend vorhanden – bedienbar, nicht nur Konsole                                                |         |           |  |
| Backend von Frontend getrennt – klare Modulgliederung                                            |         |           |  |
| Persistente Datenquelle – Datenbank oder Excel im Einsatz                                        |         |           |  |
| Mindestens eine externe API wird angesprochen                                                    |         |           |  |
| Datenverarbeitung mit pandas (oder Vergleichbarem) sichtbar                                      |         |           |  |
| README und Docstrings vorhanden                                                                  |         |           |  |
| Anwendung läuft aus dem ZIP heraus stabil (frische Installation)                                 |         |           |  |
| Datenschutz konkret berücksichtigt (keine Geheimnisse im Code, <code>.gitignore</code> sinnvoll) |         |           |  |

### Code-Qualität und Struktur (25%)

**Frage:** Ist der Code lesbar, sinnvoll benannt, modular? Ist die Schichten-Trennung erkennbar?

| Kriterium                                                               | Erfüllt | Bemerkung |  |
|-------------------------------------------------------------------------|---------|-----------|--|
| Mindestens vier Module plus <code>main.py</code>                        |         |           |  |
| Sinnvolle, sprechende Funktions- und Variablennamen                     |         |           |  |
| Trennung Frontend / Backend / Datenzugriff erkennbar                    |         |           |  |
| Funktionen haben Docstrings (Aufgabe, Parameter, Rückgabe)              |         |           |  |
| Keine doppelten Codeblöcke (DRY-Prinzip ansatzweise)                    |         |           |  |
| Fehlerbehandlung an kritischen Stellen (API, Datei, Eingabe)            |         |           |  |
| Keine Geheimnisse im Code, sondern in Konfiguration / <code>.env</code> |         |           |  |
| Lesbar formatiert (Einrückung, Leerzeichen, kein Wirrwarr)              |         |           |  |

## Dokumentation und Anforderungsanalyse (15%)

**Frage:** Passen Anforderungsanalyse und tatsächlich umgesetztes Projekt zusammen? Sind Designentscheidungen nachvollziehbar dokumentiert?

| Kriterium                                                           | Erfüllt | Bemerkung |  |
|---------------------------------------------------------------------|---------|-----------|--|
| Anforderungsanalyse liegt vor – funktional und nicht-funktional     |         |           |  |
| User Stories oder Anwendungsfälle benannt                           |         |           |  |
| Abgrenzung – was <i>nicht</i> gemacht wird – ist klar               |         |           |  |
| Datenschutz-Aspekte in der Analyse beschrieben                      |         |           |  |
| README erklärt Installation und Bedienung verständlich              |         |           |  |
| Projektdokumentation begründet die wichtigsten Designentscheidungen |         |           |  |
| Stärken, Schwächen, Potenziale sind benannt                         |         |           |  |

## Präsentation und Reflexion (20%)

**Frage:** Können Sie Ihre Anwendung verständlich vorstellen, Entscheidungen begründen und das Ergebnis realistisch einschätzen?

| Kriterium                                                       | Erfüllt | Bemerkung |  |
|-----------------------------------------------------------------|---------|-----------|--|
| Zeitraumen eingehalten (8–10 Minuten)                           |         |           |  |
| Live-Demo zeigt typische Nutzungssituation                      |         |           |  |
| Code-Walkthrough an einer sinnvoll gewählten Stelle             |         |           |  |
| Designentscheidungen können begründet werden ( ?Warum SQLite??) |         |           |  |
| Stärken realistisch und nicht überzogen benannt                 |         |           |  |
| Schwächen ehrlich benannt – ohne übertriebene Selbstkritik      |         |           |  |
| Potenziale plausibel skizziert                                  |         |           |  |
| Auf Fragen verständlich geantwortet                             |         |           |  |

## Ergänzende Beobachtungen

### Lernziele

**Besonders gut gelungen:**

---



---



---

### Wichtig!

**Empfehlung für nächste Projekte:**

---



---



---

### Praxistipp

**Was Sie weiterverfolgen sollten:**

---



---



---

*Erik Schwob – Robotron Bildungszentrum Halle*